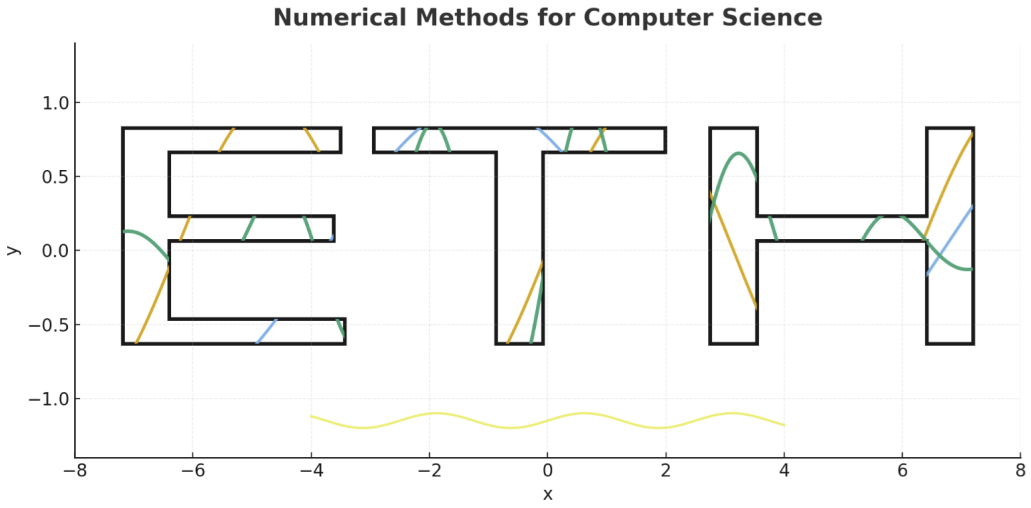

Applied Linear Algebra II — Week #11

ETH Zurich

DAVID Mihnea-Ștefan

mihdavid@ethz.ch

From exact solutions to approximate ones — this week explores how linear algebra bridges reality and computation: solving inconsistent systems through least squares and leveraging sparsity to handle massive data efficiently.

Contents

TA Award 2025

A note from your TA

Hello everyone! As you've probably already seen, VIS is once again organizing the **TA Award**, a prize given by students to the TA who resonated the most with them. If you feel that this might be me, and that I've managed, even a little, to make the NumCS course more digestible or interesting, I would be truly grateful if you considered voting for me.
On your side, it only takes a minute to open the link and cast your vote, but for me it would be a real confirmation that what I do actually has an impact. Knowing that my work has helped you, even in a small way, would mean a lot.
Thank you very much for your time and support!

1 Matrix Storage and Sparse Representations

When we move from mathematical notation to actual computation, matrices must be stored explicitly in memory. The way we store them affects both **performance** and **memory efficiency**.

1.1 Dense Matrices and Memory Layout

A dense $m \times n$ matrix requires mn scalar values. These values are stored as a single contiguous block of memory, and there are two common ways to order them:

- **Row-major order (C-style):** elements of each row are stored consecutively;
- **Column-major order (Fortran/Matlab-style):** elements of each column are stored consecutively.

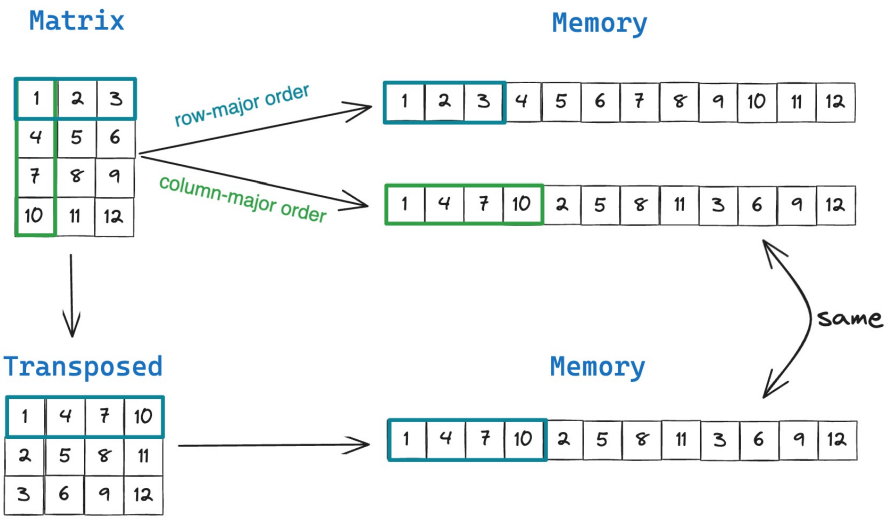

Figure 1: Row-major vs column-major memory storage and their relation to matrix transposition. This visualization clearly shows how data from a 3×3 matrix is laid out sequentially in memory in each case. Image adapted from `www.modular.com`.

> **Note**
>
> NumPy stores arrays in **row-major** format by default, while libraries such as `Fortran`, `MATLAB`, and `Eigen` use **column-major** ordering instead.
> This behavior can be controlled in NumPy using the optional `order` argument when creating or reshaping arrays:
>
> `np.array(data, order='C')` (row-major) `np.array(data, order='F')` (column-major).
>
> Internally, this affects how elements are stored and accessed in memory, which can influence cache efficiency and performance in large computations (i.e. locality).

1.2 Motivation for Sparse Matrices

In many real-world systems, circuits, networks, finite elements, web graphs, most entries of A are zero. Storing all mn values would be wasteful, so we instead store only the *non-zero entries* (denoted $\text{nnz}(A)$).

$$A \in \mathbb{R}^{m \times n} \text{ is sparse if } \text{nnz}(A) \ll mn.$$

The goal of sparse formats is simple:

- Minimize memory $\Rightarrow$ $\mathcal{O}(\text{nnz}(A))$;
- Allow fast multiplication $\Rightarrow$ $\mathcal{O}(\text{nnz}(A))$.

> Note
>
> Example: in a circuit simulation, each node connects to only a few others. The matrix of connections is mostly zero (sparse), yet structured.

1.3 Common Sparse Formats

Two classical formats are widely used to store sparse matrices efficiently, both in terms of memory and computational access patterns:

1. **Coordinate (COO) or Triplet format:** This representation stores the positions and values of all nonzero entries explicitly as triplets:

   $$A = \{(i_k, j_k, a_k)\}_{k=1}^{\text{nnz}(A)},$$

   where $\text{nnz}(A)$ is the number of nonzero elements. It is conceptually simple and ideal for constructing sparse matrices incrementally, since we can append new triplets dynamically.

   Memory usage: three arrays of size $\text{nnz}(A)$, one for row indices, one for column indices, and one for values. Hence, the storage cost is roughly:

   $$3 \times \text{nnz}(A) \text{ (numbers)}.$$

   However, COO is relatively inefficient for repeated operations because accessing or summing elements within a row requires scanning all triplets.

2. **Compressed Row Storage (CRS):** Also called **CSR**, this format compresses the row information so that the nonzero structure of each row can be accessed quickly. It uses three arrays:

   $$\texttt{val}[k], \qquad \texttt{col}[k], \qquad \texttt{rowptr}[i].$$

   The entries of the i-th row of A are found between indices `rowptr[i]` and `rowptr[i+1]` in `val`. Thus, `rowptr` contains the cumulative counts of nonzeros up to the beginning of each row.

Memory usage: CSR stores:

$$\text{nnz}(A) \text{ values} + \text{nnz}(A) \text{ column indices} + (n_{\text{rows}} + 1) \text{ row pointers.}$$

Note that the last array, `rowptr`, always has **one extra element**, a subtle but important detail often asked in the exam :)). The last entry equals $\text{nnz}(A)$, marking the end of the final row.

This extra structure makes CSR much more efficient than COO for operations like matrix–vector products or slicing by rows, since rows can be accessed directly in $\mathcal{O}(1)$ time.

Note

In practice, most numerical libraries (such as `SciPy`, `Eigen`, and `PETSc`) prefer CSR or CSC (Compressed Sparse Column) formats internally, since they balance low memory usage with fast traversal during computations.

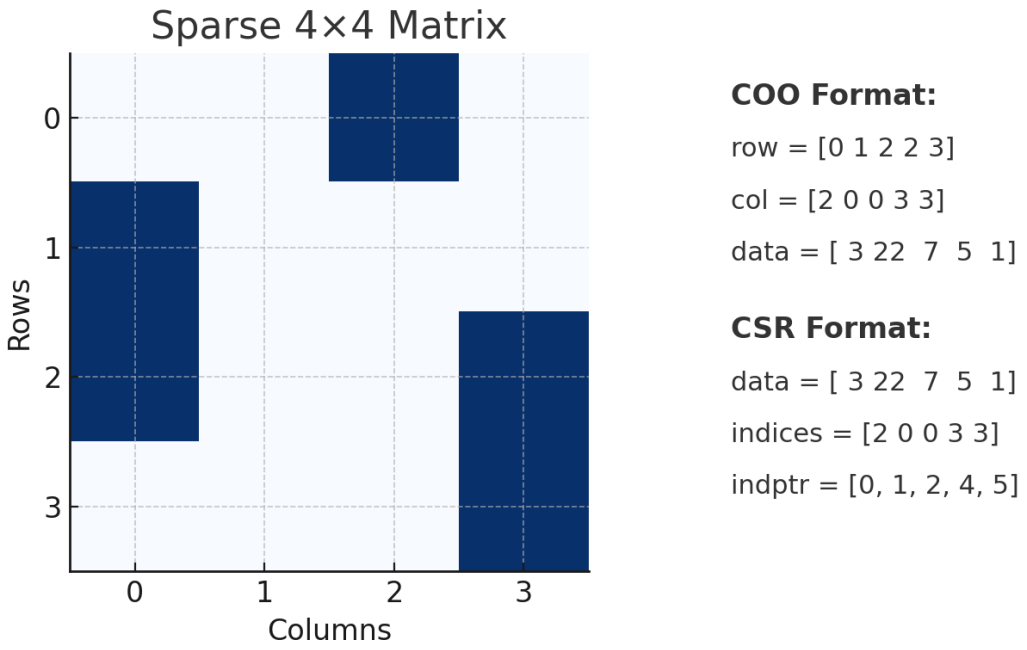

Figure 2: Comparison between **COO (Coordinate)** and **CSR (Compressed Sparse Row)** storage formats. The left panel shows nonzero positions in a 4×4 sparse matrix, while the right panel shows how values and indices are stored in memory arrays.

1.4 Sparse Matrices in Python

In Python, sparse matrices are implemented in the `scipy.sparse` module, which provides efficient data structures and operations for large, mostly-empty matrices. Rather than storing all mn entries, only the nonzero values and their locations are kept internally, following the same ideas introduced earlier (COO, CSR, CSC, etc.).

The most common constructors are:

- `coo_matrix((data, (i, j)))` — Coordinate or triplet format. Simple to create incrementally from lists or arrays of nonzero entries.
- `csr_matrix(...)` and `csc_matrix(...)` — Compressed formats optimized for numerical operations. CSR is faster for row access, while CSC is efficient for column access.
- Conversion between formats is straightforward: `A.tocsr()`, `A.tocoo()`, `A.tocsc()`.

```python
import numpy as np
from scipy.sparse import coo_matrix, csr_matrix

# ----- Constructing a sparse matrix in COO format -----
rows = np.array([0, 0, 1, 2])
cols = np.array([0, 2, 2, 1])
vals = np.array([10, 3, 5, 2])

A = coo_matrix((vals, (rows, cols)), shape=(3, 3))
print("COO format:")
print("Row indices:", A.row)
print("Col indices:", A.col)
print("Values:", A.data)

# ----- Convert to CSR (compressed format) -----
A_csr = A.tocsr()
print("\nCSR arrays:")
print("data   :", A_csr.data)
print("indices:", A_csr.indices)
print("indptr :", A_csr.indptr)

# ----- Matrix-vector product -----
x = np.array([1, 2, 3])
y = A_csr @ x
print("\nResult of A @ x:", y)
```

Note

Internally, Python uses the same CSR/CSC formats as C++ libraries such as `Eigen` . This means operations like $A\mathbf{x}$ scale with $\text{nnz}(A)$.

2 Overdetermined Linear Systems

In many applications we encounter linear systems of the form

$$Ax = b, \qquad A \in \mathbb{R}^{m\times n},\ x \in \mathbb{R}^n,\ b \in \mathbb{R}^m,$$

where the number of equations exceeds the number of unknowns:

$$m > n.$$

Such systems are called **overdetermined**. They rarely admit an exact solution, especially when the data come from measurements, experiments, or noisy real-world processes.

> Note
>
> When $m > n$, the rows of A impose more constraints than the unknown vector x can simultaneously satisfy. Unless the data are perfectly consistent (a rarity outside pure mathematics), the system $Ax = b$ has *no exact* solution.

2.1 Motivation

From Linear Algebra, a system $Ax = b$ is exactly solvable iff

$$\operatorname{rank}(A) = \operatorname{rank}([A|b]).$$

With $m > n$, this compatibility condition almost never holds.

Even if we restrict A to any n independent rows (forming a square matrix A'), we may obtain a unique solution of $A'x = b'$, but the remaining $m - n$ equations will typically be violated. Thus, instead of solving $Ax = b$ exactly, we aim to find an *approximate* solution that best fits all equations simultaneously.

This naturally leads to the least squares method, but before diving into theory, we illustrate two guiding examples.

2.2 Example 1: Linear Regression

Let the unknown affine model be

$$f(x) = a^\top x + \beta, \qquad a \in \mathbb{R}^n,\ \beta \in \mathbb{R}.$$

We have access to data points

$$(x_i, y_i) \in \mathbb{R}^n \times \mathbb{R}, \qquad i = 1, \dots, p,$$

and we want to determine the parameters (a, β).

Stacking all equations yields

$$\begin{bmatrix} x_1^\top & 1 \\ x_2^\top & 1 \\ \vdots & \vdots \\ x_p^\top & 1 \end{bmatrix} \begin{bmatrix} a \\ \beta \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_p \end{bmatrix}.$$

In reality, measurements satisfy

$$y_i = f(x_i) + \varepsilon_i,$$

where ε_i accounts for sensor noise, rounding, or random variation.

> Note
>
> In real data analysis, we *never* expect points to lie perfectly on a line or hyperplane. Noise is unavoidable, which is why least squares is fundamental in machine learning, statistics, and scientific computing.

Thus the system is inconsistent and the question becomes:

Find (a, β) that minimize the total error.

This corresponds to minimizing the squared residuals: $\sum_{i=1}^{p} \|f(x_i) - y_i\|_2^2$.

2.3 Example 2: Measuring the Angles of a Triangle

Assume multiple noisy measurements of the true angles α, β, and γ. We know the exact geometric identity:

$$\alpha + \beta + \gamma = \pi.$$

We build the augmented system:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \\ \gamma \end{bmatrix} = \begin{bmatrix} \alpha_i \\ \beta_i \\ \gamma_i \\ \pi \end{bmatrix}.$$

The first three equations represent noisy observations; the last one is an exact physical rule.

> Note
>
> Rule in data science: *never ignore exact information.* If the model guarantees $\alpha+\beta+\gamma = \pi$, then the estimate should respect it as closely as possible, even if measurements disagree.

This system is inconsistent, but we can compute the triple (α, β, γ) that best satisfies all four equations.

3 Least Squares Solutions

Having motivated why overdetermined systems arise and why exact solutions rarely exist, we now introduce the formal definition of a **least squares solution**.

3.1 Definition

Let

$$A \in \mathbb{R}^{m \times n}, \qquad b \in \mathbb{R}^m.$$

A vector $x \in \mathbb{R}^n$ is called a **least squares solution** (LSS) of the linear system $Ax = b$ if and only if

$$x \in \arg\min_{y \in \mathbb{R}^n} \|Ay - b\|_2^2.$$

> Note
>
> A least squares solution *does not need to be an actual solution* of $Ax = b$. It is simply the vector whose image Ax is closest to b in Euclidean distance.

If A has full column rank ($\text{rank}(A) = n$), then the least squares problem has a unique solution, denoted:

$$\text{LSS}(A, b) = (A^T A)^{-1} A^T b.$$

This will be justified later using normal equations and geometric projections.

> Note
>
> When A is full column rank, the matrix $A^T A$ is invertible(only in this specific case). This is why checking the rank of A is essential before applying formulas.

3.2 Example: 1-D Linear Regression

We revisit linear regression in its simplest form: fitting a line

$$f(x) = \alpha x + \beta,$$

to observed data points (x_i, y_i).

We write the system in matrix form:

$$\begin{bmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_p & 1 \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_p \end{bmatrix}.$$

Because measurements are noisy, the system is inconsistent. Hence, the parameters (α, β) must

satisfy the least squares condition:

$$(\alpha, \beta) \in \arg\min_{\tilde{\alpha},\tilde{\beta}} \sum_{i=1}^{p} \Big(y_i - (\tilde{\alpha} x_i + \tilde{\beta})\Big)^2 .$$

> **Note**
>
> The fitted line minimizes vertical squared distances between the data points and the predicted values. This is the classical interpretation used in statistics and machine learning.

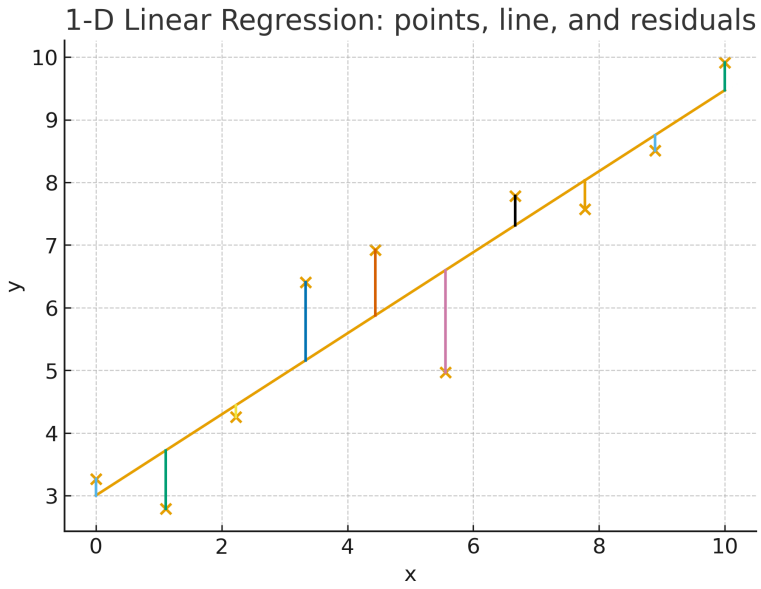

Figure 3: 1-D Linear Regression: data points, fitted line, and vertical residuals.

This example is a special case of the general least squares framework. Later, we will derive explicit formulas (normal equations), and more robust numerical methods (QR factorization).

3.3 Geometric Interpretation of Least Squares

Given an inconsistent system

$$Ax = b, \qquad A \in \mathbb{R}^{m\times n},\ m > n,$$

the vector b typically does not lie in the column space of A, denoted $\operatorname{span}(A)$.

The least squares solution $x^\star$ is defined by

$$x^\star \in \arg\min_{y\in\mathbb{R}^n} \|Ay - b\|_2^2.$$

Geometrically, $Ax^\star$ is the **orthogonal projection** of b onto the subspace $\text{span}(A)$:

$$Ax^\star = \Pi_{\text{span}(A)}(b).$$

> Note
>
> The projection always exists because $\text{span}(A)$ is a subspace. This guarantees that a least squares solution *always* exists, even when $Ax = b$ has no exact solution.

Geometric interpretation of least squares: projection of b onto span(A)

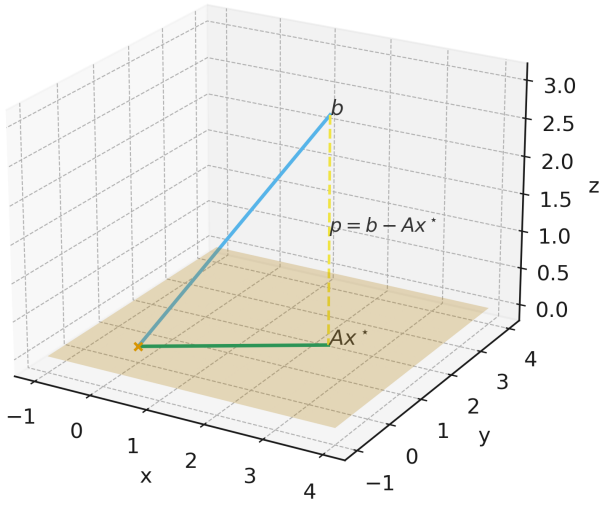

Figure 4: Geometric interpretation of least squares: the vector b and its orthogonal projection $Ax^\star$ onto $\text{span}(A)$, with residual $p = b - Ax^\star$.

Let

$$p = b - Ax^\star.$$

Because $Ax^\star$ is the orthogonal projection of b, we have

$$p \perp \text{span}(A).$$

But $\text{span}(A)$ is generated by the columns of A, so:

$$p \perp A \quad \Longleftrightarrow \quad A^T p = 0.$$

Hence:

$$A^T(b - Ax^\star) = 0.$$

Expanding gives the **normal equations**:

$$A^T A x^\star = A^T b.$$

This formula characterizes all least squares solutions.

> **Note**
>
> The normal equations come *directly* from the orthogonality condition defining projections. They are not an ad-hoc algebraic trick, but a geometric necessity.

3.4 When Does the Formula Work? Role of Full Rank

The normal equations form a square system:

$$A^T A x^\star = A^T b, \qquad A^T A \in \mathbb{R}^{n \times n}.$$

The key question:

$$\textbf{Is } A^T A \text{ invertible?}$$

Case 1 — A has full column rank ($\operatorname{rank}(A) = n$). If A has full column rank, then its columns are linearly independent. This means that the only solution of $Ax = 0$ is $x = 0$:

$$\operatorname{Null}(A) = \{0\}.$$

A key fact is that the null space of $A^T A$ is the same as the null space of A. Here is why:

$$x \in \operatorname{Null}(A^T A) \quad \Longleftrightarrow \quad A^T A x = 0.$$

Taking the scalar product with x gives

$$0 = x^T A^T A x = (Ax)^T (Ax) = \|Ax\|_2^2.$$

Thus

$$\|Ax\|_2^2 = 0 \quad \Longleftrightarrow \quad Ax = 0.$$

So we conclude

$$\operatorname{Null}(A^T A) = \operatorname{Null}(A).$$

Since $\operatorname{Null}(A) = \{0\}$, it follows that

$$\operatorname{Null}(A^T A) = \{0\}.$$

A square matrix with trivial null space is invertible, hence $A^T A$ is invertible.

$$A^T A \text{ invertible} \quad \Longrightarrow \quad x^\star = (A^T A)^{-1} A^T b.$$

Therefore the least–squares solution is **unique**.

Note

Full column rank means the columns of A form a basis for the subspace span(A) (of dimension n). Geometrically: the orthogonal projection of b onto this space lands on a unique point $Ax^\star$, and therefore the parameter vector $x^\star$ is unique.

Case 2 — A does NOT have full column rank ($\text{rank}(A) < n$**)** Then:

$$A^T A \text{ is singular (not invertible).}$$

Consequences: - the normal equations do *not* have a unique solution; == - the LS solution is not unique; - there are infinitely many minimizers.

Why? Because if $z \in \ker(A)$, then:

$$A(x^\star + z) = Ax^\star,$$

so all such vectors give the same residual norm.

Note

When $\text{rank}(A) < n$, the LS minimizer is not unique, but the projection $Ax^\star$ *is* unique. The ambiguity lies in directions that do not change the image under A.

To obtain a unique solution, one introduces the **Moore–Penrose pseudoinverse**:

$$x^\star = A^+ b,$$

which picks the minimum-norm solution among all least-squares solutions.

3.5 Minimal-Norm Least Squares Solution

When A does not have full column rank ($\text{rank}(A) = k < n$), the least squares problem

$$\min_{x \in \mathbb{R}^n} \|Ax - b\|_2^2$$

has infinitely many solutions.

The full set of least squares minimizers is an affine subspace:

$$\text{LSS}(A, b) = x_0 + \mathcal{N}(A),$$

where x_0 is any particular LS solution and $\mathcal{N}(A)$ is the null space of A.

Among these, we would like to select the one with *minimal Euclidean norm*:

$$x^\dagger = \arg\min\{\|x\|_2 \ : \ x \in \text{LSS}(A, b)\}.$$

Note

All least-squares solutions produce the same residual Ax, because adding a vector in $\mathcal{N}(A)$ does not change Ax. The minimal-norm condition identifies a single, canonical representative.

Structure of the solution space Write any LS minimizer as

$$x = x_0 + z, \qquad z \in \mathcal{N}(A).$$

We want to minimize $\|x\|_2$:

$$\|x_0 + z\|_2 \quad \text{with } z \in \mathcal{N}(A).$$

The minimum clearly occurs when $x^\dagger$ is orthogonal to $\mathcal{N}(A)$:

$$x^\dagger \perp \mathcal{N}(A).$$

Since $\mathcal{R}(A^T)$ is the orthogonal complement of $\mathcal{N}(A)$, we obtain:

$$x^\dagger \in \mathcal{R}(A^T).$$

Therefore, $x^\dagger$ must lie in the column space of A^T.

Let $V \in \mathbb{R}^{n\times k}$ be a matrix whose columns form a basis of $\mathcal{R}(A^T)$. Then every vector in this subspace has the form

$$x = Vy, \qquad y \in \mathbb{R}^k.$$

Our goal becomes finding y such that Vy satisfies the normal equations.

Derivation From the normal equations,

$$A^T Ax = A^T b,$$

and substituting $x = Vy$,

$$A^T AVy = A^T b.$$

Multiply on the left by V^T:

$$V^T A^T AVy = V^T A^T b.$$

Because V spans $\mathcal{R}(A^T)$, the matrix

$$M := V^T A^T AV \in \mathbb{R}^{k\times k}$$

is invertible. Thus,

$$y = (V^T A^T AV)^{-1} V^T A^T b.$$

Finally,

$$x^\dagger = Vy = V(V^TA^TAV)^{-1}V^TA^Tb.$$

$$\boxed{x^\dagger = V(V^TA^TAV)^{-1}V^TA^Tb}$$

This vector is the **unique minimal–norm least–squares solution**.

Moore–Penrose Pseudoinverse The expression above equals the action of the *Moore–Penrose pseudoinverse*:

$$A^\dagger := V(V^TA^TAV)^{-1}V^TA^T,$$

so that

$$x^\dagger = A^\dagger b.$$

> Note
>
> A remarkable fact: the formula of $A^\dagger$ does not depend on the particular choice of basis V for $\mathcal{R}(A^T)$. Changing V only reparametrizes y, but the product $V(V^TA^TAV)^{-1}V^T$ remains the same operator. Thus the pseudoinverse is well-defined and unique.

3.6 Optional: Moore–Penrose Pseudoinverse via SVD

*****More about SVD later**

The Singular Value Decomposition (SVD) is the most powerful and clean way to understand the structure of any matrix and to derive the pseudoinverse.

For any matrix $A \in \mathbb{R}^{m\times n}$, there exist orthogonal matrices

$$U \in \mathbb{R}^{m\times m}, \qquad V \in \mathbb{R}^{n\times n},$$

and a diagonal (rectangular) matrix

$$\Sigma \in \mathbb{R}^{m\times n}$$

such that:

$$A = U\Sigma V^T.$$

Structure of Σ. The matrix Σ contains the *singular values* of A:

$$\Sigma = \begin{pmatrix} \sigma_1 & & & & & \\ & \sigma_2 & & & & \\ & & \ddots & & & \\ & & & \sigma_r & & \\ & & & & 0 & \\ & & & & & \ddots \end{pmatrix}, \qquad r = \operatorname{rank}(A).$$

We always order the singular values as:

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0.$$

> Note
>
> The convention of ordering singular values in *strictly decreasing* order ensures uniqueness of U and V up to signs, and makes the decomposition numerically stable and conceptually clear.

Only the first r singular values are strictly positive; all others are zero.

Thus the rank of A is exactly the number of nonzero singular values.

Reduced (rank-r) SVD. We split:

$$U = [U_r \;\; U_0], \qquad V = [V_r \;\; V_0],$$

where - $U_r \in \mathbb{R}^{m\times r}$ contains the first r left singular vectors, - $V_r \in \mathbb{R}^{n\times r}$ contains the first r right singular vectors.

Define:

$$\Sigma_r = \operatorname{diag}(\sigma_1, \ldots, \sigma_r) \in \mathbb{R}^{r\times r}.$$

Then the "effective" part of A (the one carrying all the rank and geometric meaning) is:

$$A = U_r \Sigma_r V_r^T.$$

> Note
>
> The columns of V_r form an orthonormal basis of $\mathcal{R}(A^T)$, and the columns of U_r form an orthonormal basis of $\mathcal{R}(A)$. This automatically encodes both the image and the nullspace of A.

Constructing the pseudoinverse. Since Σ_r is diagonal with strictly positive entries, it is invertible and its inverse is:

$$\Sigma_r^{-1} = \operatorname{diag}\left(\frac{1}{\sigma_1}, \ldots, \frac{1}{\sigma_r}\right).$$

The Moore–Penrose pseudoinverse is obtained by "inverting" only the nonzero singular values:

$$\boxed{A^\dagger = V_r \Sigma_r^{-1} U_r^T.}$$

This formula works for *any* matrix—tall, wide, square, singular—because: - it ignores zero singular values (which cannot be inverted), - it inverts only the meaningful part of A (the rank-r

block), - it stays consistent with the geometry of projections.

Why SVD gives the minimal-norm LS solution. Let $b \in \mathbb{R}^m$. The minimal-norm least-squares solution is:

$$x^\dagger = A^\dagger b = V_r \Sigma_r^{-1} U_r^T b.$$

This vector: - lies entirely in $\mathcal{R}(A^T)$ (spanned by V_r), - ignores directions in $\mathcal{N}(A)$, - minimizes $\|x\|_2$ among all LS solutions.

> Note
>
> SVD makes it transparent why $x^\dagger$ is unique: we only use the components corresponding to the nonzero singular values. All directions along which A acts like zero are discarded.

4 Least–Squares Estimation (LSE) in Python

In practice, solving a least–squares problem is extremely simple in Python. Both NumPy and SciPy provide efficient and numerically stable tools.

We consider the standard problem

$$x^\star = \arg\min_x \|Ax - b\|_2^2,$$

where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

4.1 Using NumPy: `numpy.linalg.lstsq`

The recommended function for solving LSE is `np.linalg.lstsq`, which internally uses an SVD-based algorithm (stable and robust).

```python
import numpy as np

# Example design matrix A (m > n)
A = np.array([[1, 1],
              [2, 1],
              [3, 1],
              [4, 1]], dtype=float)

# Observations b
b = np.array([2.2, 2.8, 3.6, 4.5])

# Least-squares solution using SVD under the hood
x_star, residuals, rank, svals = np.linalg.lstsq(A, b, rcond=None)

print("Solution x* =", x_star)
print("Residual vector =", A @ x_star - b)
print("Rank(A) =", rank)
print("Singular values =", svals)
```

Note

`np.linalg.lstsq` is the preferred method for LSE. It automatically handles rank-deficient matrices and returns the minimal–norm LS solution.

4.2 Using the pseudoinverse: `numpy.linalg.pinv`

NumPy provides a direct implementation of the Moore–Penrose pseudoinverse. This is equivalent to computing

$$x^\dagger = A^\dagger b.$$

```
A_pinv = np.linalg.pinv(A)    # SVD-based pseudoinverse
x_dagger = A_pinv @ b

print("Minimal-norm LS solution =", x_dagger)
```

Note

When A is rank-deficient, `pinv` automatically returns the *unique minimal-norm LS solution* using the SVD formula $A^\dagger = V_r \Sigma_r^{-1} U_r^T$.

4.3 Normal equations (not recommended in practice)

The least–squares solution satisfies the *normal equations*:

$$A^T A x = A^T b.$$

Formally,

$$x^\star = (A^T A)^{-1} A^T b,$$

but **computing the inverse explicitly is discouraged**.

A better approach is to solve the linear system directly using `np.linalg.solve`, which is faster and more numerically stable than forming an inverse.

```
# Normal equations: solve (A^T A)x = A^T b

ATA = A.T @ A
ATb = A.T @ b

# Avoid np.linalg.inv(ATA); solve the linear system instead:
x_normal = np.linalg.solve(ATA, ATb)

print("Solution via normal equations (solve) =", x_normal)
```

Note

Using `solve` is always preferable to `inv`, because solving a system is more accurate and significantly more efficient than computing a matrix inverse.
However, the normal–equation method still squares the condition number:

$$\kappa(A^TA) = \kappa(A)^2,$$

so it can be highly unstable for ill-conditioned matrices. This is why `lstsq` and `pinv` (both SVD-based) remain the recommended methods in scientific computing.

5 Pitfalls of normal equations and how to avoid them

The method based on the normal equations

$$A^TAx = A^Tb$$

is simple to derive but can be problematic in practice, both numerically and computationally. Here we highlight two major issues.

5.1 1. Numerical instability (roundoff errors)

Even if A has full column rank, the matrix A^TA may lose rank *numerically*. Small perturbations in A become squared in A^TA.

For example, if

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1+\varepsilon \end{bmatrix}, \qquad |\varepsilon| \leq \varepsilon_{\text{machine}},$$

then

$$A^TA = \begin{bmatrix} 2 & 2+\varepsilon \\ 2+\varepsilon & 1+(1+\varepsilon)^2 \end{bmatrix} \approx \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix},$$

which is numerically rank deficient, even though A itself has rank 2. Thus A^TA may become noninvertible due to floating-point roundoff.

Note

This is the reason why solving LS via A^TA is discouraged. QR or SVD methods avoid these instabilities.

Safer alternatives

- **QR factorization**: $A = QR$, then $x^\star$ solves $Rx = Q^Tb$. Fast and stable.
- **Cholesky** (only if A^TA is well-conditioned and SPD): $A^TA = LL^T$, then solve $LL^Tx = A^Tb$.

- **Pivoted LU**: More robust than plain LU, can handle nearly singular matrices.
- **SVD**: Most stable, gives minimal-norm solution. Recommended when A is close to losing rank.

5.2 2. Loss of sparsity (fill-in effect)

Even if A is sparse, the product A^TA is typically *not* sparse. Many zero entries become nonzero (a phenomenon known as *fill-in*). This makes storage and computations expensive for large systems.

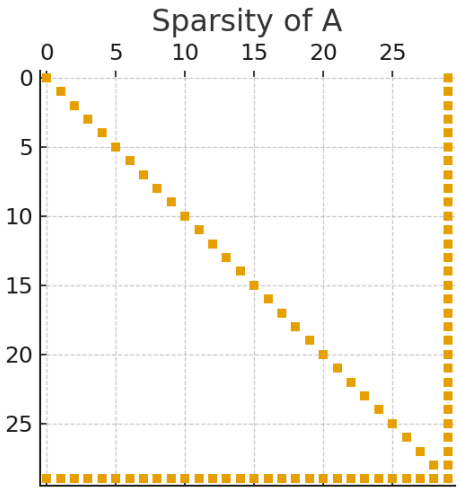

Figure 5: *
Sparsity of A

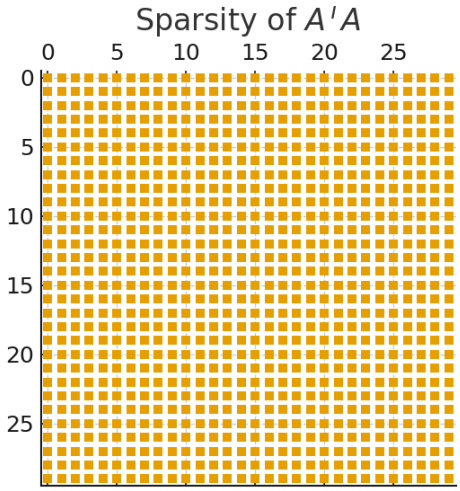

Figure 6: *
Sparsity of A^TA

Figure 7: Comparison: sparse A vs. dense A^TA due to fill-in.

> **Note**
>
> In large-scale scientific computing, avoiding the formation of A^TA is a key design principle, especially for sparse matrices.

5.3 A trick to avoid computing A^TA

The normal equation form

$$A^T(Ax - b) = 0$$

can be written as a *saddle–point system*. Let $r = Ax - b$ be the residual. Then

$$A^Tr = 0, \qquad r = Ax - b.$$

This leads to the augmented sparse system

$$\begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} r \\ x \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix}, \qquad \in \mathbb{R}^{(m+n)\times(m+n)}.$$

- If A is sparse, the block matrix above is also sparse.
- No product A^TA is formed.
- Suitable for iterative solvers (CG, MINRES, LSQR).

Note

This formulation is widely used in large scientific applications (e.g. PDE-constrained optimization). It preserves sparsity and avoids the ill-conditioning of A^TA.